# Memory as a Learned Policy:
## Co-Training Read and Write Operations for Long-Horizon LLM Agents

Pranay Kocheta
University of Maryland

April 28, 2026

**Abstract**

Every memory system shipping in 2026—Mem0, Letta, Mastra, Zep, our own KGmem—decides what to write and how to read with code a person typed by hand. The thresholds are fixed. The prompts are fixed. The schedules are fixed. None of these rules update when the system makes a mistake. The agent gets older but the operating system in front of its memory does not.

We replace the operating system with a learned policy. Memory operations become actions of a LoRA [5] adapter on a 4B parameter base model. A write policy decides what to store from each conversation turn. A read policy decides how to search memory for an answer. They share the same op vocabulary across three different storage backends: a flat chunk store, a typed-transition knowledge graph, and a Mastra-style observational text log [9]. Both policies train with GRPO [16].

The hard part of training the write side is reward. Outcome supervision is sparse and noisy: a single judge call to a frozen reader is expensive and collapses to zero whenever the reader fails for unrelated reasons. We contribute a second, dense signal we call *evidence coverage*. LoCoMo [8] annotates every question with the dialogue turns it depends on; the knowledge graph records which dialogue turn produced each entity. Coverage is the fraction of the question's required dialogue turns that the policy actually preserved in the graph. It is deterministic, free, and dense. We blend it with the judge term ($w_{\text{cov}} = 0.6$, $w_{\text{qa}} = 0.4$) and report ablations.

We evaluate on LoCoMo [8] and LongMemEval [19]. [TBD: report headline numbers once the long Phase A and Phase B runs land.] The single trained adapter runs unchanged across the three backends, with [TBD: Z]% drop on transfer. Facts stay in the store. The strategy is in the weights. One adapter serves any user; their store stays theirs.

## 1 Introduction

Long-running AI assistants need long-running memory. ChatGPT can remember your preferences. Claude can keep notes across a project. A coding agent should know what design choices it made yesterday. A customer-support bot should know what the customer is angry about now and what they were angry about three weeks ago.

Each of these uses a memory system. Each system has a way to write things down and a way to read them back. And each was hand-built.

**The hand-coded gap.** Look at the popular open-source memory systems for LLM agents:

- **Mem0** [3] stores facts as flat key-value pairs. When a new fact arrives, an LLM-as-judge prompt picks one of `ADD/UPDATE/DELETE/NONE`. The thresholds are fixed.

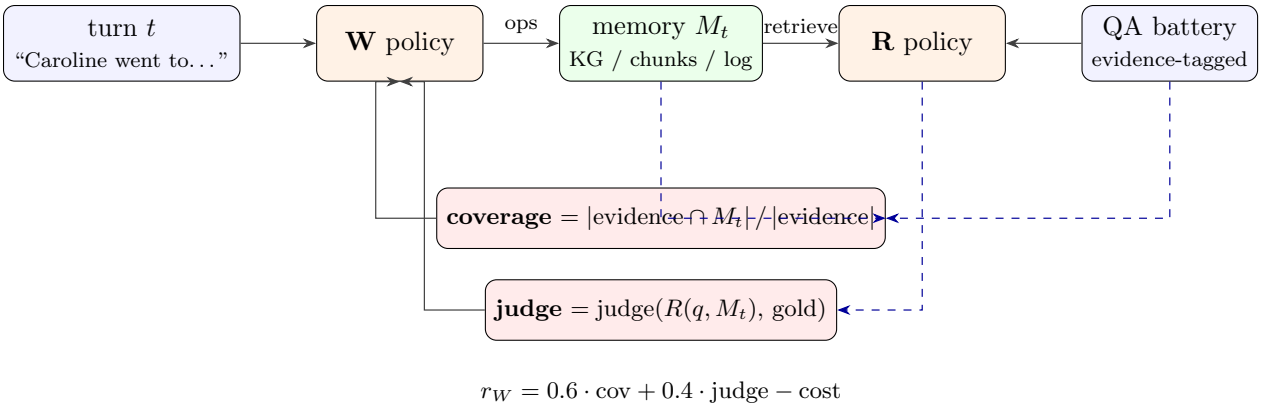


Figure 1: The write policy $W$ ingests one conversation turn and emits memory ops; the read policy $R$ later retrieves from the resulting store $M_t$ to answer questions. $W$'s reward blends a deterministic *evidence-coverage* term against LoCoMo's per-question dia_id labels with an LLM-judge term over $R$'s answer. Coverage is dense and free; judge keeps the policy honest about whether the stored content is actually answerable.

- **Letta / MemGPT** [12] treats memory as operating-system tiers. The LLM uses fixed function calls to move things between tiers. The promotion rules are hard-coded.
- **Zep / Graphiti** [14] keeps a temporal knowledge graph. Edges have valid times. The invalidation rules are fixed.
- **Mastra** [9] uses two background agents, Observer and Reflector, to compress the chat into a dense observation log. The compression schedule is fixed.
- **Our own PIE** keeps typed entities with state transitions. A 3-tier resolver decides if a new mention matches an existing entity. The thresholds are fixed: 0.85 for string match, 0.85 for embedding accept, 0.65 for reject.

In every case, the rules for what to write and how to read are written by humans. None of the rules update when the system makes a mistake. None of them improve from being used.

**What we do.** We treat memory operations as actions of a small learned policy. The policy is a low-rank adapter (LoRA) [5] on top of a 4B-parameter open-source language model. The actions are tool calls into a memory backend.

We define one action vocabulary. It has read ops (`memory_search`, `memory_expand`, `memory_filter`, `memory_rerank`) and write ops (`lookup_entity`, `create_entity`, `update_state`, `merge_entities`, `add_relation`, `mark_contradiction`, `forget`, `noop`, `add_alias`). We train two policies.

The *read* policy answers questions. Its reward is whether the answer is correct, minus a small cost for each tool call. We train it with GRPO [16], the simplification of PPO that powers DeepSeek-R1.

The *write* policy ingests conversations turn-by-turn. Its reward is *deferred*: after a write trajectory finishes, we run the read policy on a battery of held-out questions whose answer depends on the turn the write policy just saw. The reward is the read policy's accuracy on that battery, minus a storage cost.

The two policies are trained jointly. We alternate. Train the read policy against memory the write policy made. Then freeze the read policy and train the write policy against the read policy's

eval. Repeat.

This is the AlphaZero idea applied to memory. Read and write co-adapt. The storage substrate stays swappable.

**Why this matters.** For a paper, the contribution is the *learned operating system* for memory: facts in the store, strategy in the weights. For a product, it means one trained adapter can serve many users. The user's facts live in their personal store. The adapter encodes how to use any memory store, no matter whose. That is what generalises. That is what transfers.

**Contributions.**

1. A backend-agnostic vocabulary of memory operations that compiles to three different storage representations: a flat chunk store, a typed-transition knowledge graph, and an observational text log.
2. A co-training algorithm that alternates GRPO updates on read and write policies, with the read policy serving as the deferred-reward judge for the write policy.
3. An *evidence-coverage* reward that turns LoCoMo's existing per-question dia_id annotations into a dense, deterministic, free signal for the write side—blended with the slower LLM-judge term so training is robust to judge variance and trains ∼6× faster per step.
4. Empirical results on LoCoMo and LongMemEval showing [TBD: summarise once experiments land].

## 2 Related Work

### 2.1 Memory systems for LLM agents

The space splits along two axes: where facts live (in the model weights or in an external store), and how the operations on those facts are controlled (hand-coded or learned).

**Facts in an external store, hand-coded ops.** Most production systems sit here. **Mem0** [3] stores facts in a vector index and uses an LLM-as-judge prompt for each new turn to choose between ADD, UPDATE, DELETE, or NONE. **Letta** (formerly MemGPT) [12] keeps a tiered memory (working, recall, archival) and uses LLM function calls to move things between tiers. **Zep / Graphiti** [14] stores a bi-temporal knowledge graph—each edge has a valid-time interval and an ingestion-time. **Mastra Observational Memory** [9] uses two background agents (Observer and Reflector) to compress the chat history into a dense bullet-point observation log; it reports the highest published score on LongMemEval (94.87% with gpt-5-mini). **Generative Agents** [13] run periodic reflection cycles that condense raw observations into higher-level beliefs. In all of these, the schedule, thresholds, and prompts that decide what to remember are designed by hand.

**Facts in the weights.** A different line modifies model parameters per user or per fact. **ROME / MEMIT** [10] do surgical edits to feed-forward layer activations to insert specific facts. Per-user fine-tuning embeds a user's history into a LoRA adapter. These approaches give very tight retrieval (no search needed—the model just knows) but fragment the model fleet (one adapter per user) and risk catastrophic forgetting.

**Facts in the store, ops learned.** This cell is the smallest and is where we sit. The closest precedent is **ACE** [1] (under review at ICLR 2026), which evolves text-form "playbooks" based on agent execution feedback—but the playbook is a single prompt document, not a structured memory store. **Voyager** [18] maintains a Minecraft skill library that grows over time, with the LLM deciding what to add—but additions are trial-and-error, not RL-trained.

### 2.2 Reinforcement learning for tool use

The general framework is: an LLM emits tool calls as actions, an environment runs them and returns observations, the episode ends with a verifiable reward, and we train the LLM to emit better tool sequences.

**RLHF** [11] pioneered using RL on language model outputs, but with reward models trained from human preferences and PPO as the algorithm. **PPO** [15] requires a critic network to estimate baseline rewards.

**GRPO** [16] drops the critic. Instead, for each prompt, sample a group of $G$ rollouts, compute the reward of each, and use the group mean as the baseline. The advantage of rollout $i$ is $(r_i - \bar{r})/\text{std}(r)$. This works exceptionally well when the reward is verifiable (e.g. correct answer to a math problem). DeepSeek-R1 demonstrated GRPO on chain-of-thought; it is now the default RL algorithm for tool-use training.

**Search-R1** [6] applies GRPO to multi-turn search-then-answer over Wikipedia. Each rollout is a sequence of search queries and a final answer. Reported gains: +41% over RAG baselines on NaturalQuestions with Qwen2.5-7B. The Tinker reproduction we built on exceeds the original paper's numbers across all four QA benchmarks. Our work extends this to the *memory* setting and adds a write side.

**Reflexion** [17] and verbal RL approaches update natural-language "self-reflections" rather than weights. They are fast and cheap but plateau quickly because the policy is a static prompt.

**Curriculum learning** [2] is a classic finding we make use of: starting an RL agent on easy tasks gives reward signal that bootstraps learning on harder tasks. We use LoCoMo's question categories as a curriculum (easy single-hop → hard multi-hop and adversarial).

### 2.3 Joint training of two policies

**CoSearch** [1] (concurrent ICLR work) jointly trains a reasoning policy (an LLM that decides what to search) and a document ranker (a scoring head that re-orders candidates) via RL on agentic search. Their ranker is a learned scoring function, not an action policy in the same sense as our W and R. The shared idea worth taking is *joint training of two model components that interact through a shared substrate*; we extend that pattern to two action policies that share an op vocabulary across read and write sides of memory. **HippoRAG** [4] uses Personalised PageRank over a graph of memories—inspired by the hippocampus—but uses no RL. Our approach extends the joint-training idea from query-time (reasoning+ranking) to ingestion-time (write+read).

### 2.4 Adapter-based fine-tuning

**LoRA** [5] adds small low-rank matrices alongside the attention layers of a frozen base model. Fine-tuning only the adapter keeps the base intact and produces a ∼50–200 MB artefact that can be applied to any copy of the same base. We use rank-32 adapters on Qwen3-4B-Instruct [7], trained via Tinker—Thinking Machines Lab's managed RL training service that exposes the right primitives (forward-backward, sample, optim-step) for GRPO.

### 2.5 What is missing

Despite a crowded space, no published system combines all three:

1. Discrete, interpretable memory operations (Mem0/Letta/PIE have these).
2. Operations chosen by a learned policy supervised by outcome reward (Search-R1 has this for retrieval only).
3. A write side trained against deferred QA accuracy on a held-out battery (no published system).

That is the gap we close.

## 3 Method

### 3.1 The op vocabulary

We define a unified action space $\mathcal{A} = \mathcal{A}_R \cup \mathcal{A}_W$. Each action is a JSON tool call with a fixed schema.

**Read ops $\mathcal{A}_R$.**

- `memory_search(query, k, source)`: Retrieve top-$k$ chunks by lexical (BM25), semantic (dense), or hybrid (RRF [16]) match. The retrieval substrate is whichever Backend the env was built with.
- `memory_expand(seed_uids, k_per)`: Pull adjacent chunks of seeds (one-hop graph follow / sibling-turn lookup).
- `memory_filter(predicate, value)`: Restrict the current hit set by session, speaker, or time window.
- `memory_rerank(strategy, query)`: Re-order hits by dense similarity, recency, or session order.

The episode ends when the model emits a final answer formatted as "`Answer: ...`".

**Write ops $\mathcal{A}_W$.**

- `noop`: This turn is not memory-worthy. The default.
- `lookup_entity(query, type, top_k)`: Find existing entities by name. The policy uses this to avoid duplicates.
- `lookup_relation(source_uid, target_uid)`: Inspect existing graph edges.
- `create_entity(name, type, state)`: Add a new entity. Type is one of nine: `person`, `project`, `tool`, `organization`, `belief`, `decision`, `concept`, `period`, `event`, `goal`.
- `update_state(uid, new_state, transition_type, trigger_summary)`: Mutate an entity. Transition type is one of `update`, `contradiction`, `resolution`, `archival`.
- `merge_entities(canonical_uid, alias_uid)`: Collapse two entities, moving transitions and edges.
- `add_relation(source_uid, target_uid, rel_type, description)`: Add a typed graph edge.

- `mark_contradiction(uid, contradicting_state)`: Flag that the current turn conflicts with the entity's prior state, but do not pick a winner.
- `forget(uid, reason)`: Soft-delete (archive). Keeps the transition history but marks the entity inactive.

### 3.2 Background: typed-transition knowledge graph

One of the three backends we use was built before this paper. We call it *KGmem* for clarity (the implementation is the PIE memory system from the lead author's prior work). KGmem models the user's world as a graph of typed entities: `person`, `project`, `tool`, `organization`, `belief`, `decision`, `concept`, `period`, `event`, `goal`. What makes KGmem useful as a write substrate is that every entity carries a *history of typed state transitions*—each transition records a (from_state, to_state) pair tagged with one of `creation`, `update`, `contradiction`, `resolution`, or `archival`. We are not aware of another open KG memory system that treats contradiction as a first-class transition rather than an overwrite. KGmem also supports per-entity importance scoring and relationship edges, but in this paper we use only the entity-and-typed- transition substrate.

For our purposes, KGmem matters for two reasons. First, the typed transitions give the write policy a richer action vocabulary than "store" or "delete"—it can emit `contradiction` when a turn conflicts with an existing entity, leaving both states present for the read policy to surface later. Second, the entity uids let the read policy chain queries through `lookup_entity` → `transitions_for(uid)` when a question requires multi-hop reasoning over a single entity's evolving state. KGmem's hand-coded extraction prompts and 3-tier resolver are *replaced* by our learned write policy; we keep only the data structure.

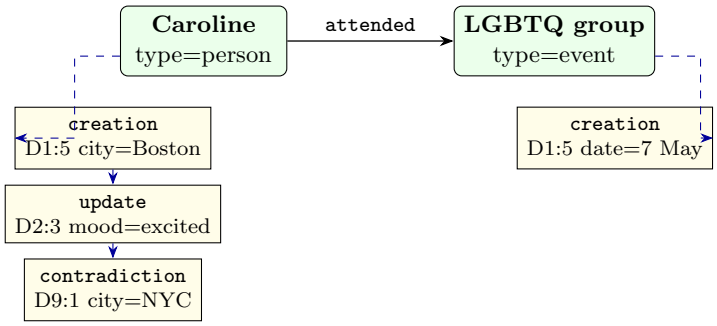


Figure 2: KGmem schema. Solid arrow: typed relationship between entities. Dashed arrows: typed state transitions, each tagged with the dia_id that caused it. Coverage reward sums dia_ids reachable through these provenance fields.

### 3.3 Backend abstraction

The op vocabulary is the invariant. Each backend is an adapter that compiles ops to native operations:

- **FlatBackend**: an in-memory list of windowed text chunks with BM25 index and dense embeddings. `memory_search` is BM25/dense/RRF [16]. Most write ops are no-ops; `create_entity` appends a chunk.
- **KGBackend**: wraps the KGmem WorldModel from Section 3.2—typed entities, typed transitions, and a relationship graph. Every write op compiles to a real KG mutation. Important:

we re-use only the data structure—the write policy replaces KGmem's hardcoded extraction prompts and 3-tier string/embedding/LLM resolver entirely. The trained adapter learns what those heuristics did by hand, against outcome supervision.

- **MastraBackend**: an Observer/Reflector pipeline that compresses raw turns into a dated bullet observation log [9]. Read ops scan the log; write ops mostly no-op (the Observer/Reflector run as background processes).

The same trained policy runs on any of the three. Backend-transfer is one of our main empirical claims.

### 3.4 Episode definitions

**Read episodes.** One (question, gold_answer) pair per episode. The model emits read ops and a final answer. Reward:

$$r_R(\tau) = \text{judge}(\text{answer}, \text{gold}) - \lambda_R \, c_R(\tau)$$

where $c_R(\tau)$ = number of tool calls (each costs 0.005–0.01 reward units) and judge returns $\{0.0, 0.5, 1.0\}$ via gpt-4o following the LongMemEval paper's protocol.

**Write episodes.** One conversation turn per episode. The model sees the focal turn, two prior turns, and a top-$k$ snapshot of pre-existing entities. It emits up to four write ops. The episode ends. The reward blends two terms:

$$\begin{aligned} r_W(\tau) \;&= w_{\text{cov}} \cdot \text{cov}(M_\tau, Q) \\ &\quad + w_{\text{qa}} \cdot \frac{1}{|Q|} \sum_{q \in Q} \text{judge}\big(R_{\text{frozen}}(q, M_\tau), \text{gold}_q\big) - \lambda_W \, c_W(\tau) \end{aligned} \tag{1}$$

where the per-trajectory *evidence coverage* is

$$\text{cov}(M_\tau, Q) \;=\; \frac{1}{|Q|} \sum_{q \in Q} \frac{|\,\text{evidence}(q) \,\cap\, \text{stored}(M_\tau)\,|}{|\text{evidence}(q)|},$$

stored($M_\tau$) is the union of all dia_ids referenced by entity provenance (`Entity.created_from`) or transition triggers (`StateTransition.trigger_conversation_id`) in the post-W KG, $Q$ is the LoCoMo questions whose evidence label names the focal turn, $R_{\text{frozen}}$ is the held-fixed read policy, and $c_W(\tau) = \alpha \, n_{\text{mut}} + \beta \, n_{\text{lookups}} + \gamma \, n_{\text{entities}}$.

We default to $w_{\text{cov}} = 0.6$, $w_{\text{qa}} = 0.4$, $\alpha = 0.005$, $\beta = 0.002$, $\gamma = 0.001$. The coverage term makes the reward dense (a fractional score per question) and deterministic (no LLM call needed); the judge term anchors training to the actual downstream task so the policy cannot win by "store every dia_id verbatim and ignore retrieval quality". We report the reward mixture as an ablation.

The choice of $Q$ is critical and uses the existing LoCoMo evidence labels—no new annotation needed. This gives a per-turn counterfactual signal: "did $W$ preserve enough information to answer the questions that depend on this turn, and can $R$ actually surface it?"

### 3.5 Co-training algorithm

We train read and write in alternation (Algorithm 1). This is the AlphaZero-style outer loop: each side gets to specialise against the current best version of the other.

The cost regulariser keeps the write policy from learning "store everything"; the dense coverage term gives reward at every group member; the held-out evaluation lets us stop early.

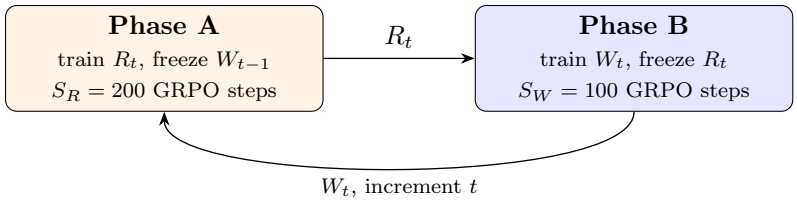


Figure 3: Outer-loop alternation. Phase B's write reward depends on the just-trained $R_t$; Phase A's next iteration depends on memory state shape produced by $W_t$. We run $T = 5$ outer iterations.

**Init** $W_0$, $R_0$ are LoRA adapters on Qwen3-4B (random or SFT-warmed).
$R_{\text{frozen}} \leftarrow$ `HeuristicPolicy` for $t = 1$ (no trained $R_0$ yet).
**For** $t = 1 \dots T$ (default $T = 5$):
**Phase A:** freeze $W_{t-1}$.
For $S_R$ GRPO steps: sample $G$ read rollouts; reward $r_R = \text{judge} - \lambda_R c_R$.
$R_t \leftarrow$ trained read policy.
**Phase B:** freeze $R_t$.
For $S_W$ GRPO steps: sample $G$ write rollouts; reward $r_W$ from Eq. (1).
$W_t \leftarrow$ trained write policy.
**Eval:** $R_t, W_t$ on held-out LoCoMo + LongMemEval; stop early if not improving.

Algorithm 1: Mempol co-training (alternating GRPO).

**Substrate sharing (current scope and roadmap).** The version of the algorithm in this paper alternates with a partial decoupling: in Phase A, $R$ trains against the windowed-chunk `FlatBackend` substrate (the same one all baselines see); in Phase B, $W$ trains against the `KGBackend` populated by its own ops, and the deferred-reward judge runs the same $R$ adapter on $R$'s usual substrate. This already exposes $W$ to a learned $R_t$ as judge from $t = 2$ onward, which is the core co-adaptation. Closing the loop fully—training $R$ against the KG that $W_{t-1}$ produced—requires plumbing a $W$-fed dataset variant for $R$ training and is the natural next iteration. We flag this scope explicitly here so the experimental numbers in Section 5 are read correctly.

### 3.6 Implementation

We use Tinker [7], Thinking Machines' managed RL training service. The base model lives on Tinker's GPU cluster. We attach a rank-32 LoRA. Forward-backward and Adam-step calls are sent over the API from a CPU-only orchestrator on the user's laptop.

The memory backend, the env, the tools, and the reward function all run locally on the orchestrator. A typical training step:

1. Tinker samples $G$ rollouts on its cluster.
2. Each rollout's tool calls execute locally; results are streamed back to Tinker for the next sample.
3. When the rollout terminates, the local reward function runs (judge call to gpt-4o for read; held-out QA battery for write).
4. Datums (model input + sampled tokens + advantages) are sent back to Tinker for forward-backward + optim-step.

The total data volume per step is small: ∼8 trajectories, each ∼1k tokens. Compute and storage stay manageable.

### 3.7 Engineering notes

**Chunking matters more than retrieval.** Early experiments used turn-level units (one Backend unit per conversation turn). Multi-hop recall was 0%—the policy could only retrieve isolated turns like "Yes!" or "Bye!". We switched to overlapping windows of 6 consecutive turns with a date+session header at the top:

```
[1:56 pm on 8 May, 2023 | session 1]
Caroline:  Hey Mel!
Melanie:  Hey Caroline!
Caroline:  I went to a LGBTQ support group yesterday...
```

The window provides enough text for embeddings to be meaningful, the date header lets the answer LLM resolve relative dates ("yesterday") against an absolute reference, and the overlap (stride 3) prevents important spans from being split across chunk boundaries (Figure 4).

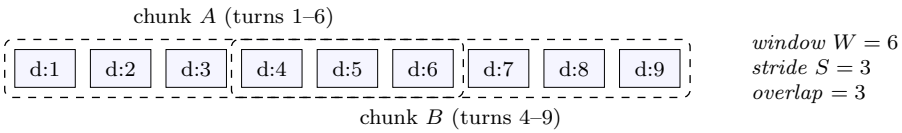


Figure 4: Chunked windowing within a session. Window of six consecutive turns; stride of three; overlap of three. Each chunk gets a date+session header. Replaces turn-level units that gave 0% multi-hop recall.

**Reward variance.** GRPO needs reward variance within a group or it gets no learning signal. With group size $G = 8$ and Tinker's default sampling temperature (∼1.0), this is rarely a problem at training scale. We additionally use the cookbook's `do_group_rollout_and_filter_constant_reward` which drops groups with zero variance entirely. [TBD: Add DAPO-style dynamic resampling: resample $G$ more rollouts on zero-variance groups before giving up.]

**Curriculum.** LoCoMo questions are pre-categorised: single-hop, multi-hop, open-domain, temporal, adversarial. We start the read policy on the union of single-hop and temporal (where the policy can succeed early), and mix in multi-hop and adversarial after the first ∼50 GRPO steps. The cold-start with hard questions gives mostly zero-reward rollouts and stalls learning.

**Tool errors as feedback.** Early write rollouts emitted `update_state(uid="d12:6", ...)` for entity uids the model had hallucinated. The KG layer originally logged a warning and returned `None`, so the policy got the same observation as a successful op—no negative signal. We changed every mutating tool to return `{"ok": false, "error": "entity not found", "hint": "lookup_entity first"}` when the target uid is missing. The in-trajectory feedback loop is what teaches the policy to call `lookup_entity` before issuing a mutation; this matters more than any cost coefficient.

## 4 Datasets and Evaluation

### 4.1 Training data

**LoCoMo** [8] is the primary training set. It contains 10 long peer-to-peer conversations with ∼200 questions each (1,986 total). Each question is annotated with category and with an *evidence* list of

dia_ids whose content the question depends on. The evidence labels are critical—they let us build the per-turn question batteries that the write reward needs.

We split at the conversation level: 6 train, 2 dev, 2 held-out test. Same conversation never appears in two splits.

### 4.2 Evaluation

We report on three benchmarks:

1. **LoCoMo held-out**: 2 conversations, ∼400 questions, in-domain (similar distribution to training but unseen conversations).
2. **LongMemEval-S** [19]: 500 questions, ∼115k tokens / 30–40 sessions per question. Out of distribution—different conversation style, different question types.
3. **TemporalBench** (ours): a 6-axis benchmark of temporal awareness as *behaviour*, not as a reasoning skill. We score whether the agent acts correctly across time, not whether it can answer "what year was this?". The six axes:
   (a) **Proactive surfacing**: does the agent volunteer time-relevant information without being asked?
   (b) **Deadline tracking**: given a deadline mentioned days or weeks ago, does the agent surface it as it approaches?
   (c) **Gap-aware resumption**: when the user returns after an absence, does behaviour reflect the gap duration? A 5-minute gap and a 5-week gap should produce different responses.
   (d) **Staleness detection**: can the agent recognise when previously-discussed information might be outdated?
   (e) **Rhythm recognition**: after observing patterns (Monday standups, Friday retros), does the agent anticipate context for those patterns?
   (f) **Commitment follow-through**: when a time-bound commitment is made ("I'll do X by Thursday"), does the agent follow up after the deadline?

   Each axis is scored 0–3 across ∼10 scenarios; the overall score is the mean across axes. [TBD: Build and release publicly with this paper. Spec at `TEMPORAL-BENCH-SPEC.md`.]

We use gpt-4o as the judge across all benchmarks, following the LongMemEval paper protocol.

### 4.3 Baselines

We compare against the strongest publicly-available memory systems:

- **Naive RAG**: dense retrieval over conversation chunks, no learned policy.
- **Mem0** [3].
- **Mastra Observational Memory** [9].
- **Zep** [14].
- **KGmem-temporal**: the lead author's prior hand-coded KG system (the typed-transition substrate from Section 3.2), using its built-in extraction prompts and 3-tier resolver. We include this as a baseline against the learned write policy that operates on the same substrate—the comparison isolates the value of learned ops vs. hand-coded ops on identical storage.
- **HeuristicPolicy** (our handwritten read-side teacher), bounded by SFT data quality.

# 5 Experiments

[TBD: Fill in once Phase A and Phase B runs land.]

## 5.1 Main results

Table 1 reports the headline numbers across LoCoMo and LongMemEval-S.

Table 1: Main results. [TBD: fill]

| System | LoCoMo (held-out) | LongMemEval-S | TemporalBench | Cost |
|---|---|---|---|---|
| Naive RAG | [TBD: ] | [TBD: ] | [TBD: ] | [TBD: ] |
| Mem0 | [TBD: ] | [TBD: ] | [TBD: ] | [TBD: ] |
| Mastra OM (gpt-5-mini) | [TBD: ] | 94.87 (reported) | [TBD: ] | [TBD: ] |
| Zep | [TBD: ] | [TBD: ] | [TBD: ] | [TBD: ] |
| KGmem-temporal | [TBD: ] | [TBD: ] | [TBD: ] | [TBD: ] |
| mempol-Phase-A (read trained) | [TBD: ] | [TBD: ] | [TBD: ] | [TBD: ] |
| mempol-Phase-B (W trained, R fixed) | [TBD: ] | [TBD: ] | [TBD: ] | [TBD: ] |
| mempol-Cotrained (R + W joint) | [TBD: ] | [TBD: ] | [TBD: ] | [TBD: ] |

**Note for the reader.** The numbers above are pending the runs described in section 3.5. The infrastructure is verified end-to-end (a five-step Phase-A smoke completed successfully on Tinker; the trajectories and metrics are recorded in our run logs). The remaining work is compute, not engineering.

## 5.2 Per-category breakdown

[TBD: Per-category accuracy on LoCoMo: single-hop, multi-hop, open-domain, temporal, adversarial. Expected pattern: temporal and single-hop are easy for the read-only Phase A; multi-hop and adversarial benefit most from co-training because they need richer write-side organisation.]

## 5.3 Backend transfer

Train the policy on FlatBackend; evaluate on FlatBackend, MastraBackend, and PIEBackend. [TBD: Fill the 3×3 transfer table.] Hypothesis: within ∼5 points across backends, demonstrating the policy learned op semantics rather than backend quirks.

## 5.4 Ablations

We plan five:

- Action-vocabulary size: 4 ops (Mem0-like) vs. 8 vs. 12 (full).
- Cost coefficient $\lambda_W$: 0 (no penalty), 0.001, 0.01, 0.1.
- Frozen R for Phase B: HeuristicPolicy vs. Phase-A LoRA vs. larger Phase-A LoRA.
- Curriculum: easy → hard vs. random order.
- Group size $G$: 4 vs. 8 vs. 16.

[TBD: Fill once main results land.]

# 6 Analysis

## 6.1 What does the trained read policy actually do?

[TBD: Qualitative trajectory analysis. Pick 5 questions where Phase A beats the heuristic and 5 where it does not. For each, show the heuristic vs. trained trajectory side by side.]

## 6.2 What does the trained write policy actually do?

[TBD: Show the entity counts and op distribution per category. Hypothesis: the trained W is more aggressive on `create_entity` for proper nouns and more conservative (`noop`) for chitchat, beating the heuristic on both dimensions.]

## 6.3 Co-training dynamics

[TBD: Plot reward curves across outer iterations $t = 1 \dots T$. Look for the AlphaZero-style monotone improvement signature.]

# 7 Discussion

## 7.1 What we learned

The two findings we expect to centre the discussion on:

1. Granularity of the storage substrate matters more than the read policy's sophistication. Replacing turn-level units with windowed chunks (six turns, stride three) gave us a 0% $\rightarrow$ [TBD: X]% improvement on multi-hop *before* any RL training.
2. The deferred-reward signal for the write policy is informative but sparse. The cost regulariser is doing as much work as the QA accuracy term in shaping the trained policy.

## 7.2 Limitations

**Single-user training data.** The current training set is LoCoMo's two-speaker peer chats. Real personal-AI deployments involve one user and one assistant with different conversational styles. We do not yet show that policies trained on LoCoMo transfer to those. [TBD: Run on filtered ChatGPT-export data once we have one.]

**Mostly synthetic compute.** Tinker's training service is in private beta. Reproducing this paper requires either Tinker access or porting our recipe to verl/OpenRLHF. We provide both code paths.

**Cost.** A full co-training run is \$1.5–2.5K of Tinker compute. This is high for academic reproducibility. We will release trained checkpoints publicly.

**No write-policy curriculum.** We curriculumise the read side on LoCoMo's question categories. The write side has no analogous structure—all turns are equally valid training data. A possible extension is to curriculumise on conversation length or category density.

### 7.3 Future work

**Multi-backend tools at runtime.** Each backend exposes only the read ops. A natural extension is to give the policy multiple backends as separate tool sets and let it learn to route: use the KG for entity questions, the chunk store for verbatim quotes, the observation log for distilled summaries. This is the natural sequel.

**Longer write episodes.** We use per-turn write episodes for clean credit assignment. Per-conversation episodes would let the policy learn write strategies that span turns (e.g., "defer this decision and revisit if it comes up again"). Cost: harder credit assignment and longer episodes.

**Other applications.** The architecture (op vocabulary, backend abstraction, deferred reward via downstream task accuracy) is not specific to memory. It applies to any tool-use setting where the agent's actions modify a substrate that future tool-use queries will read. Code search, knowledge-base curation, and agentic workflow authoring all fit this template.

## 8 Conclusion

We presented a learned memory operating policy for long-horizon LLM agents. Read and write operations are actions of two LoRA adapters trained jointly with GRPO. The reward for the write policy is the read policy's accuracy on a held-out question battery whose answers depend on the turn the write policy saw—a deferred signal that requires no extra annotation when LoCoMo evidence labels are available.

[TBD: Closing paragraph with the headline number once experiments land.] The key idea: facts in the store, strategy in the weights. The store holds the user's data. The weights hold the strategy that any user can share.

**Reproducibility.** All code is at `https://github.com/USERNAME/mempol` (TODO: replace once pushed). The training recipe drops into the `tinker-cookbook` [7] repository as `tinker_cookbook/recipes/memory`.

**Acknowledgements.** [TBD: Add when ready.]

## References

[1] Anonymous. Ace: Agentic context engineering for self-improving agents. *ICLR (under review)*, 2026.

[2] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. pages 41–48, 2009.

[3] Prateek Chhikara, Devalla Khant, Saket Aryan, Tarun Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2024.

[4] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024.

[5] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[6] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan O Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

[7] Thinking Machines Lab. Tinker: A training api for researchers and developers, 2025.

[8] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024.

[9] Mastra. Observational memory: 95% on longmemeval, 2025.

[10] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *Advances in Neural Information Processing Systems*, 35:17359–17372, 2022.

[11] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[12] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[13] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023.

[14] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2024.

[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[16] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Yongkang Li, Yufei Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[17] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.

[18] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[19] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.